

Semana 4: implementação e teste isolado da Hash Unit

Projeto: CI Digital Inatel, T25S2, Grupo 2. Responsável por esta entrega: Samuel. Versão preparada em 8 de outubro de 2026, para a primeira implementação funcional.

O módulo foi compilado e simulado: os 181 vetores de referência passaram, com comparação do CRC, da chave e de todos os metadados. O objetivo deste guia é explicar como construir e verificar essa primeira versão, para que Samuel possa apresentar as decisões ao grupo.

1. Entenda a tarefa desta semana.

A distribuição informada por Samuel é: Herman implementa o Packet Parser; Bruno implementa a Flow Identification Unit; Samuel implementa a Hash Unit. A sua parte começa quando uma `flow_key` já formada chega à interface do Hash.

O texto principal de `Semana 4.docx` pede explicitamente implementar a Hash Unit e testá-la isoladamente com resultados previamente conhecidos. O anexo geral coloca o Hash na Semana 5, mas informa que não substitui as orientações específicas de cada semana. Esta entrega segue a orientação principal recebida: uma primeira versão funcional e testável do Hash.

Orientação da Semana 4	Como foi atendida
Criar RTL com a interface do contrato	Módulo <code>hash_unit</code> , com as 13 portas previstas.
Receber a chave e os metadados	Captura conjunta de chave, comprimento e timestamp.
Implementar a função de hash	CRC-16/CCITT-FALSE aplicado aos 13 bytes da chave.
Gerar e disponibilizar o índice	Saída <code>hash_index_out[15:0]</code> , junto aos dados associados.
Respeitar a validade	Entrada e saída com transferência por <code>valid && ready</code> .
Associar cada entrada ao seu resultado	Um registro por vez; todos os campos ficam preservados.
Testar isoladamente com valores conhecidos	Referência independente em Python, arquivo de vetores e <code>testbench</code> .

2. Fixe o que entra e o que sai.

A Hash Unit transforma um registro de 200 bits de dados em um registro de 216 bits de dados. Os 16 bits adicionais são o índice. Clock, reset, validade e disponibilidade não entram nessa soma.

Porta	Direção em relação ao Hash	Bits	Função
<code>clk</code>	Entrada	1	Clock; transferências na borda de subida.
<code>reset_n</code>	Entrada	1	Reset síncrono, ativo baixo, adotado nesta versão.
<code>flow_key_in</code>	Entrada	104	Chave formada pela Flow Identification Unit.
<code>packet_length_in</code>	Entrada	16	Comprimento associado ao pacote.
<code>timestamp_in</code>	Entrada	80	Timestamp associado ao pacote.
<code>in_valid</code>	Entrada	1	O produtor apresenta um registro válido.

Porta	Direção em relação ao Hash	Bits	Função
in_ready	Saída	1	O Hash pode aceitar esse registro.
hash_index_out	Saída	16	CRC completo da chave.
flow_key_out	Saída	104	A mesma chave recebida.
packet_length_out	Saída	16	O mesmo comprimento recebido.
timestamp_out	Saída	80	O mesmo timestamp recebido.
out_valid	Saída	1	Índice e dados associados estão disponíveis.
out_ready	Entrada	1	O consumidor pode receber o resultado.

Na conexão com Bruno, a saída FlowID.out_valid alimentará Hash.in_valid; Hash.in_ready retornará a FlowID.out_ready. No lado do Controller, Hash.out_valid alimentará Controller.in_valid; Controller.in_ready retornará a Hash.out_ready. Os dados correspondentes seguem o mesmo caminho.

Comprimento e timestamp acompanham o resultado, mas somente a flow_key participa do CRC. Portanto, dois pacotes com a mesma chave produzem o mesmo índice, mesmo com comprimento ou timestamp diferentes. Esses pacotes continuam sendo dois registros distintos.

3. Defina o CRC sem deixar parâmetros implícitos.

Escrever apenas “CRC-16” não determina um resultado único. O contrato da Semana 3 define o índice de 16 bits, mas deixa a variante e sua realização para a implementação. Os relatórios de pesquisa do grupo recomendam o perfil abaixo, adotado como base desta primeira versão.

Parâmetro	Valor adotado
Nome	CRC-16/CCITT-FALSE, também denominado CRC-16/IBM-3740
Largura	16 bits
Polinômio	0x1021, correspondente a $x^{16} + x^{12} + x^5 + 1$
Inicialização de cada chave	0xFFFF
Reflexão da entrada	Não
Reflexão da saída	Não
XOR final	0x0000
Quantidade processada	Exatamente 13 bytes
Ordem dos bytes	Do byte mais significativo da chave ao menos significativo
Índice produzido	Todos os 16 bits do CRC final

Esta escolha registra um perfil reproduzível para implementar e testar o módulo; não transforma a recomendação dos relatórios em aprovação conjunta das equipes. O código, a referência em software e os valores esperados usam exatamente o mesmo perfil documentado.

Os relatórios antigos também discutiam selecionar cinco bits para uma tabela de 32 posições. O README e o contrato atual adotam o CRC completo de 16 bits. Por isso, esta versão não aplica

aquela seleção antiga. Um índice de 16 bits representa 65.536 valores possíveis, de 0x0000 a 0xFFFF.

4. Confira a ordem dos 104 bits antes de calcular.

A chave é a concatenação unidirecional da 5-tuple:

```
flow_key = {src_ip, dst_ip, src_port, dst_port, protocol};
```

Campo	Posição na chave	Bytes processados
IP de origem	[103:72]	0 a 3
IP de destino	[71:40]	4 a 7
Porta de origem	[39:24]	8 e 9
Porta de destino	[23:8]	10 e 11
Protocolo	[7:0]	12

Dentro de cada campo com mais de um byte, processamos primeiro o byte mais significativo. Assim, o primeiro byte do Hash é `flow_key_in[103:96]` e o último é `flow_key_in[7:0]`. A ordem das posições de bytes do barramento MAC já foi resolvida na extração e na formação da chave; ela não deve ser aplicada novamente à chave entregue ao Hash.

Exemplo: TCP de 192.168.1.10:50000 para 192.168.1.20:443.

Campo	Representação hexadecimal
IP de origem	C0A8010A
IP de destino	C0A80114
Porta de origem, 50000	C350
Porta de destino, 443	01BB
Protocolo TCP, 6	06

A chave completa é 104'hC0A8010AC0A80114C35001BB06. Com o perfil adotado, o índice esperado é 16'hB511.

5. Capture o registro inteiro antes de começar.

Uma entrada é aceita em uma borda de subida somente quando `reset_n = 1`, `in_valid = 1` e `in_ready = 1`. Nesse momento, o RTL copia a chave, o comprimento e os 80 bits do timestamp para registradores internos.

Essa captura conjunta é o que mantém a associação correta: o Hash calcula a partir da chave armazenada e entrega os metadados armazenados na mesma aceitação. Ele não continua consultando os dados externos enquanto calcula. O testbench altera dados de entrada inválidos depois de uma aceitação para conferir esse comportamento.

Se `in_valid = 1` e `in_ready = 0`, o produtor deve manter validade e dados até a aceitação. Se `out_valid = 1` e `out_ready = 0`, o Hash deve manter validade, índice, chave, comprimento e timestamp até a entrega. Um `valid` mantido durante espera não representa várias transferências.

Seguindo a orientação do professor Elivander, não há FIFO nesta primeira versão. Existe armazenamento para um registro em processamento ou aguardando entrega. Registradores para preservar esse registro são necessários mesmo sem uma fila de vários pacotes.

6. Implemente o cálculo de um byte por ciclo.

Abra `rtl/hash_unit.sv`. A função `crc16_next_byte` recebe o CRC acumulado e um byte. Ela executa a seguinte transformação:

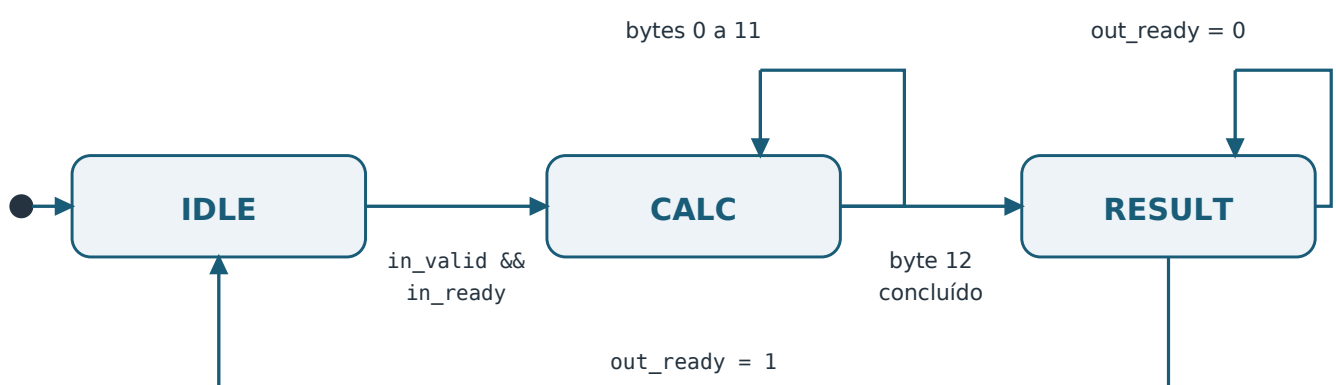
```
value = crc XOR (byte deslocado 8 posições para a esquerda)
repetir 8 vezes:
  se o bit 15 de value for 1:
    value = (value deslocado 1 posição à esquerda) XOR 0x1021
  caso contrário:
    value = value deslocado 1 posição à esquerda
conservar a largura de 16 bits
```

O for de oito repetições descreve lógica combinacional para processar os oito bits de um byte. Ele não cria oito ciclos de clock. A máquina de estados chama essa transformação para um novo byte a cada ciclo, totalizando 13 atualizações do CRC por chave.

O CRC começa novamente em `0xFFFF` para cada entrada aceita. Esse reinício é necessário para impedir que o resultado do pacote anterior afete o seguinte.

O controlador usa três estados:

Estado	Comportamento
IDLE	Ativa <code>in_ready</code> com <code>reset</code> inativo e aguarda uma entrada válida. Ao aceitar, captura o registro, inicializa o CRC e entra em <code>CALC</code> .
CALC	Mantém <code>in_ready = 0</code> , processa um byte e atualiza o CRC. Após o byte 12, guarda o CRC final e entra em <code>RESULT</code> .
RESULT	Apresenta <code>out_valid = 1</code> . Mantém todos os dados até <code>out_ready = 1</code> em uma borda de subida; depois retorna a <code>IDLE</code> .



O reset síncrono tem prioridade sobre esses estados: na borda com `reset_n = 0`, cancela o registro pendente e reinicializa o controle. Transferências não são contabilizadas durante reset.

No último byte, observe a atribuição `result_reg <= next_crc`. Usar `crc_reg` nesse ponto entregaria o acumulado anterior ao processamento do 13º byte, porque atribuições não bloqueantes atualizam os registradores ao final da etapa de simulação da borda.

Para o exemplo C0A8010AC0A80114C35001BB06, a referência independente calcula:

Byte	Valor hexadecimal	CRC após esse byte
Inicialização	-	FFFF
0	C0	38BC
1	A8	3FB9
2	01	6E9D
3	0A	B122
4	C0	4CB6
5	A8	0BAA
6	01	0B4A
7	14	A9DE
8	C3	13EC
9	50	94A7
10	01	741C
11	BB	34A3
12	06	B511

7. Separe resultado disponível de resultado entregue.

Considere E0 a borda em que uma entrada é aceita. A sequência da implementação é:

Borda	Acontecimento
E0	Captura dos metadados e inicialização do CRC.
E1 a E12	Processamento dos bytes 0 a 11.
E13	Processamento do byte 12. Logo após a borda, o resultado está disponível e out_valid sobe.
E14, no caso mais rápido	Primeira entrega possível, se out_ready = 1.
E15, no caso mais rápido	Primeira aceitação possível do próximo registro.

Portanto, são 13 ciclos entre a aceitação e a disponibilidade do resultado. A primeira entrega ocorre 14 ciclos após a aceitação, e o menor intervalo entre aceitações é 15 ciclos nesta implementação. Não há aceitação de uma nova entrada na mesma borda de entrega da anterior.

Se o consumidor permanecer indisponível, RESULT dura mais tempo. Essa espera aumenta o intervalo entre entradas, sem alterar o índice ou os metadados apresentados.

Esses números descrevem o RTL simulado. A folga relatada pelo professor sustenta a escolha inicial sem FIFO, mas o atendimento do caminho completo também depende das esperas dos outros módulos. A frequência alcançável em FPGA precisa de síntese e análise temporal na etapa correspondente.

8. Calcule valores esperados sem reutilizar o código do RTL.

Abra `python/reference_hash.py`. A referência usa a função da biblioteca padrão `binascii.crc_hqx(key, 0xFFFF)`, com a chave de 13 bytes na mesma ordem documentada. O resultado não vem da função `SystemVerilog` do módulo testado.

Primeiro, a referência se confere com a sequência ASCII 123456789, cujo CRC esperado para esse perfil é 0x29B1. Essa sequência tem nove bytes e serve apenas para conferir o perfil do modelo. Ela não é aplicada diretamente à porta de 104 bits do RTL: acrescentar quatro bytes mudaria o CRC esperado.

Depois, o script confere os exemplos dos relatórios do grupo e gera todos os resultados esperados antes da simulação. Para inspecionar a geração, execute na pasta do pacote:

```
python3 python/reference_hash.py
```

Caso	Chave de 104 bits, em hexadecimal	Índice esperado
TCP do exemplo da FI	C0A8010AC0A80114C35001BB06	B511
Mesmo TCP, outros metadados	C0A8010AC0A80114C35001BB06	B511
Sentido inverso do exemplo	C0A80114C0A8010A01BBC35006	EB45
UDP, porta de destino 53	0A0000010A0000023039003511	890D
Referência da pesquisa, destino .32	0A000020C0000220C00001BB06	83A4
Referência da pesquisa, destino .33	0A000020C0000221C00001BB06	C604
Referência da pesquisa, destino .34	0A000020C0000222C00001BB06	08E4
Referência da pesquisa, destino .35	0A000020C0000223C00001BB06	4D44
Exemplo da pesquisa do Parser	C0A8010A08080808CB8401BB06	A018
Treze bytes zero	0000000000000000000000000000	280C
Treze bytes FF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FED3
ASCII de 13 bytes, 123456789ABCD	31323334353637383941424344	36CC
Chave distinta com colisão no exemplo	C0A8010BC0A80114C350B9DA06	B511

Os dois casos com chaves distintas e índice B511 mostram por que a chave completa precisa acompanhar o hash. O índice não identifica um fluxo de maneira única. A Hash Unit entrega ambas as chaves corretamente; a comparação com a Flow Table pertence ao Controller.

Os casos de zeros, FF, ASCII e bits isolados exercitam a entrada de 104 bits como um vetor. Eles não representam, necessariamente, metadados de pacotes válidos. Interpretar e validar pacotes é responsabilidade das etapas anteriores.

A coleção contém 13 casos dirigidos, 104 chaves com um único bit ativo e 64 casos pseudoaleatórios com semente fixa 392: 181 vetores. Os timestamps incluem bits acima da posição 63 e o limite de 80 bits, permitindo detectar truncamento indevido.

`tb/hash_vectors.csv` permite ler os casos e os valores esperados. `tb/hash_vectors.mem` contém o mesmo conjunto no formato de 216 bits lido pelo testbench:

Campo no arquivo .mem	Bits
flow_key	[215:112]

Campo no arquivo .mem	Bits
packet_length	[111:96]
timestamp	[95:16]
CRC esperado	[15:0]

Cada linha tem 54 dígitos hexadecimais. Essa organização do arquivo é uma convenção do teste; não adiciona uma porta de 216 bits ao módulo.

9. Teste o protocolo junto com o CRC.

Abra `tb/tb_hash_unit.sv`. O módulo testado é instanciado sozinho; o testbench representa o produtor e o consumidor. Ao aceitar uma entrada, o teste guarda seu resultado esperado em uma estrutura de conferência, denominada scoreboard. Ao entregar uma saída, compara o registro completo de 216 bits com o esperado.

Verificação	Erro que permite detectar
Resultado após as 13 atualizações	Byte omitido, resultado antecipado ou cálculo que não termina.
Chave repetida, metadados diferentes	Falta de reinício do CRC ou mistura de registros.
Todas as posições da chave exercitadas	Bit ignorado, ordem errada ou largura incorreta.
Alteração de dados de entrada inválidos	Uso contínuo da entrada em vez do registro capturado.
Entrada mantida enquanto <code>in_ready = 0</code>	Aceitação indevida ou duplicação de registro.
Resultado mantido enquanto <code>out_ready = 0</code>	Perda de validade ou alteração dos dados durante espera.
Reset durante cálculo	Resultado de operação cancelada reaparecendo.
Reset com saída pronta e bloqueada	Entrega residual de registro anterior ao reset.
Chaves distintas com o mesmo CRC	Descarte indevido ou perda da chave original.
Contagem e comparação de todas as entregas	Duplicação, saída sem entrada, troca de ordem ou corrupção de metadados.

A fila de valores esperados existe apenas no testbench. Ela não é uma FIFO adicionada ao hardware. O RTL continua armazenando um único registro por vez.

Na saída, a estrutura compara o CRC e todos os 200 bits associados. Verificar somente o índice não detectaria, por exemplo, um timestamp incorreto entregue junto de um CRC correto.

10. Execute e interprete o resultado.

Pré-requisitos da execução validada: Python 3.10 ou superior, Verilator com suporte a `--binary` e `--timing`, compilador C++ e make. O Python usa somente sua biblioteca padrão. Na pasta extraída `semana4_hash`, execute:

```
python3 python/run_tests.py --simulator verilator
```

Se o compilador não estiver no PATH, informe seu caminho real:

```
python3 python/run_tests.py --simulator verilator --tool /caminho/para/verilator
```

O script gera os valores esperados, compila o RTL e o testbench, executa a simulação e grava os registros em `validation/`. Ele encerra com erro se a compilação falhar, uma comparação divergir ou os testes não terminarem com a confirmação final.

O script também possui uma opção para Icarus Verilog, exigindo `iverilog` e `vvp`. Essa opção não foi executada nesta entrega; a validação registrada foi feita com Verilator.

Resultado confirmado em 8 de outubro de 2026:

```
PASS: latencia de 13 atualizacoes e saida preservada na espera.
PASS: reset durante calculo, sem resultado residual.
PASS: reset com resultado em espera, sem entrega residual.
PASS: 181 vetores golden, CRC e todos os metadados conferidos.
SUMMARY accepted=184 delivered=182 reset_cancelled=2 pending=0
SUMMARY output_stability_checks=272 input_wait_cycles=2785
PASS: tb_hash_unit concluido sem divergencias.
```

A coleção de 181 vetores foi aplicada inteira. Antes dela, o teste aceitou três registros dirigidos: entregou um e cancelou dois por reset. Por isso existem 184 aceitações e 182 entregas. A conta fecha: $184 = 182 + 2 + 0$ pendentes. Os cancelamentos foram provocados pelo teste, de acordo com o comportamento de reset.

A compilação final não apresentou avisos do Verilator. Também passou a conferência estática do RTL com `--lint-only -Wall`. O clock de 10 ns usado pelo testbench é um estímulo de simulação e não comprova funcionamento físico em FPGA nessa frequência.

11. Apresente os arquivos e as decisões ao grupo.

Arquivo	O que apresentar
<code>rtl/hash_unit.sv</code>	Interface, captura conjunta, função CRC, estados e retenção do resultado.
<code>python/reference_hash.py</code>	Cálculo independente e geração reproduzível dos valores esperados.
<code>tb/tb_hash_unit.sv</code>	Estímulos isolados e comparação automática de índice e metadados.
<code>python/run_tests.py</code>	Comando para reproduzir a compilação e o teste.
<code>tb/hash_vectors.csv</code>	Relação legível dos 181 casos conhecidos antes da simulação.
<code>tb/hash_vectors.mem</code>	Valores esperados usados pelo testbench.
<code>validation/resultados.md</code>	Ambiente, resultados e alcance da validação.
<code>validation/simulation.log</code>	Registro da simulação executada.
Este guia	Explicação das decisões e do processo.

Uma explicação objetiva para a reunião é: “Implementei uma primeira Hash Unit com CRC-16/CCITT-FALSE. Ela recebe a chave de 104 bits, calcula o índice completo de 16 bits e preserva comprimento e timestamp de 80 bits. Trabalha com um registro por vez, sem FIFO, usando `valid/ready`. O teste isolado comparou 181 vetores com uma referência independente, além de verificar esperas, associação dos dados e cancelamento por reset.”

Para a conexão com os demais módulos, esta versão torna explícitos o perfil do CRC, a ordem dos bytes, o timestamp de 80 bits, o clock comum e o reset síncrono ativo baixo. O contrato da Semana 3 registra alguns desses itens como propostas de alinhamento. A simulação comprova o comportamento da Hash Unit sob essas escolhas; a compatibilidade final depende de os módulos conectados adotarem o mesmo contrato.

A unidade trata o timestamp como um vetor preservado, sem interpretar unidade de tempo ou realizar operações sobre ele. O significado de `packet_length` também vem do produtor; o Hash não recalcula esse comprimento. Esses limites evitam atribuir ao Hash responsabilidades do Parser, da FI ou da atualização de features.

O pacote entrega a primeira versão funcional pedida nesta semana. Os resultados registrados são de verificação isolada: não constituem medições de área, frequência máxima ou desempenho do subsistema integrado.

As fontes utilizadas foram o arquivo `Semana 4.docx` recebido nesta conversa, o [README do projeto](#), o [contrato revisado da Hash Unit](#), os relatórios de pesquisa de Samuel e as referências técnicas primárias abaixo. O repositório foi consultado no estado da árvore `bd485ba5899386b0dc7c45bab289ab5a143b6f17`.

- [Relatório final de pesquisa da Hash Unit](#), blob `3cba511c1aa2beab505d86a4f6aea47b46aafbce`: perfil CRC e ordem de processamento; seleção antiga de bits não adotada nesta versão.
- [Relatório de integração Parser-Hash](#), blob `5c693aa9c068c2f42333a9f934e0bb71bd25e6cb`: exemplos e processamento da 5-tuple.
- [Comparativo CRC-16 e seleção de bits](#), blob `88427ddc8a4f0388483819cb84f7306ff5015bdc`: contexto da proposta anterior.
- [CRC RevEng: CRC-16/IBM-3740, alias CRC-16/CCITT-FALSE](#): parâmetros do perfil e valor de conferência `0x29B1`.
- [Python: `binascii.crc\_hqx`](#): função utilizada pelo modelo de referência, polinômio `0x1021` e valor inicial informado pelo chamador.
- [Verilator: opções de execução](#): compilação com `--binary` e suporte temporal com `--timing`.